UTS

PEMBELAJARAN MESIN



Nama	: Nabil Mukhlas Priyo Santoso
NIM	: 202431105
Kelas	: E
Nama Dosen	: Rizqia Cahyaningtyas, ST., M.Kom
No. Bangku	: 14
Nama Asisten	:

1. Muhammad Caisar Gemilang Pratama
2. Benedict Samuel Ardy

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2026/2027

DAFTAR ISI

BAB I PENDAHULUAN **3**
1.1 Rumusan Masalah 3
1.2 Tujuan Masalah 3
1.3 Manfaat Masalah 4
BAB II LANDASAN TEORI **5**
2.1 Pembelajaran Mesin dan Klasifikasi 5
2.2 Exploratory Data Analysis (EDA) 5
2.3 Preprocessing Data 5
2.4 Decision Tree 6
2.5 K-Nearest Neighbors (KNN) 6
2.6 Evaluasi Model 7
2.7 Penelitian Terkait 7
BAB III HASIL **9**
3.1 Pemahaman Dataset 9
3.2 Exploratory Data Analysis 10
3.3 Preprocessing Data 14
3.4 Pelatihan dan Hasil Decision Tree 15
3.5 Pelatihan dan Hasil KNN 18
3.6 Perbandingan Model dan Analisis 19
BAB IV PENUTUP **22**
4.1 Kesimpulan 22
4.2 Saran 22
DAFTAR PUSTAKA **23**

BAB I
PENDAHULUAN

1.1 Rumusan Masalah

Dataset yang digunakan pada UTS ini adalah data permainan Counter-Strike 2 (berkas 03_counter_strike_2.csv) yang berisi 2.000 baris dan 14 atribut. Setiap baris merekam kondisi sesaat dalam sebuah ronde, mulai dari sisa waktu, skor kedua tim, peta, status bom, total nyawa (health), armor, uang, hingga jumlah helm, dengan label akhir berupa tim pemenang ronde (round_winner: CT atau T). Berdasarkan data tersebut, rumusan masalah yang dibahas adalah sebagai berikut.

1. Bagaimana karakteristik dan pola data pada dataset Counter-Strike 2 berdasarkan hasil Exploratory Data Analysis (EDA)?
2. Tahapan preprocessing apa yang diperlukan agar data siap digunakan oleh algoritma Decision Tree dan K-Nearest Neighbors (KNN)?
3. Bagaimana performa Decision Tree dan KNN dalam memprediksi tim pemenang ronde (round_winner) yang diukur menggunakan akurasi, precision, recall, dan F1-score?
4. Algoritma manakah yang memberikan hasil terbaik dan faktor (fitur) apa yang paling berpengaruh terhadap hasil prediksi?

1.2 Tujuan Masalah

Tujuan yang ingin dicapai dari praktikum UTS ini adalah:

1. memahami struktur, distribusi, dan hubungan antar atribut pada dataset melalui EDA;
2. melakukan preprocessing data, meliputi pemeriksaan data kosong dan duplikat, encoding data kategorikal, pembagian data latih dan data uji, serta standarisasi fitur;
3. melatih model Decision Tree dan KNN pada data latih dan menghitung akurasi pada data uji;
4. membandingkan hasil kedua model serta menganalisisnya berdasarkan teori dan penelitian pendukung.

1.3 Manfaat Masalah

Manfaat yang diperoleh dari praktikum ini adalah pemahaman menyeluruh mengenai alur kerja klasifikasi pada pembelajaran mesin, mulai dari eksplorasi data, preprocessing, pelatihan model, hingga evaluasi. Praktikum ini juga memberikan gambaran bagaimana pemilihan algoritma, penskalaan fitur, dan penyetelan hyperparameter memengaruhi akurasi model. Selain itu, hasil

analisis dapat menjadi contoh penerapan machine learning untuk memprediksi hasil pertandingan berdasarkan kondisi permainan, yang relevan dalam analisis performa pada esports.

BAB II
LANDASAN TEORI

2.1 Pembelajaran Mesin dan Klasifikasi

Pembelajaran mesin (machine learning) adalah cabang kecerdasan buatan yang memungkinkan komputer mempelajari pola dari data tanpa diprogram secara eksplisit untuk setiap aturan. Pada pendekatan supervised learning, model dilatih menggunakan data yang telah memiliki label sehingga mampu memprediksi label data baru. Klasifikasi merupakan salah satu tugas utama supervised learning, yaitu proses mencari pola atau fitur yang membedakan kelas-kelas data untuk memprediksi kelas objek yang labelnya belum diketahui (Dwinnie dkk., 2024). Pada dataset yang digunakan, target round_winner hanya memiliki dua nilai (CT dan T) sehingga permasalahan ini termasuk klasifikasi biner.

2.2 Exploratory Data Analysis (EDA)

Exploratory Data Analysis adalah tahap awal analisis untuk memahami data sebelum pemodelan, yaitu menelaah sekumpulan data agar dapat dipahami, menemukan hubungan yang tidak terduga, serta merangkumnya (Dwinnie dkk., 2024). Kegiatan EDA umumnya mencakup pemeriksaan tipe data dan data kosong, statistik deskriptif, visualisasi distribusi (histogram dan boxplot), analisis distribusi kelas target, serta analisis korelasi antar fitur. Distribusi kelas penting diperiksa karena ketidakseimbangan kelas dapat membuat akurasi tampak tinggi walaupun kelas minoritas sulit dikenali. Sementara itu, outlier dideteksi dengan aturan Interquartile Range (IQR), yaitu nilai di bawah $Q1 - 1{,}5 \times IQR$ atau di atas $Q3 + 1{,}5 \times IQR$.

2.3 Preprocessing Data

Preprocessing adalah tahap pembersihan dan transformasi data mentah menjadi bentuk yang sesuai bagi algoritma pembelajaran. Tahap ini meliputi penanganan data tidak konsisten, pembersihan noise, penghapusan data duplikat, serta konversi tipe data, misalnya atribut bertipe teks diubah menjadi bilangan agar dapat diproses model (Dwinnie dkk., 2024; Putri dkk., 2024). Atribut kategorikal tanpa urutan, seperti nama peta, lazim diubah dengan one-hot encoding sehingga setiap kategori menjadi kolom biner tersendiri.

Tahap penting lainnya adalah pembagian data menjadi data latih dan data uji. Data latih dipakai untuk membangun model, sedangkan data uji dipakai untuk mengukur kemampuan generalisasi pada data yang belum pernah dilihat. Rasio 80:20 banyak digunakan dalam penelitian serupa (Dwinnie dkk., 2024; Allorerung dkk., 2024). Pembagian secara stratified menjaga proporsi kelas pada kedua subset tetap sama.

Selain itu, normalisasi atau standarisasi fitur diperlukan ketika skala antar fitur berbeda jauh. Tanpa penskalaan, algoritma berbasis jarak hanya bergantung pada fitur berskala besar, sedangkan fitur berskala kecil menjadi tidak berpengaruh (Allorerung dkk., 2024). Salah satu metode yang umum adalah Z-Score (standarisasi), yang mengubah setiap nilai berdasarkan rata-rata dan simpangan baku fitur:

$$z = (x - \mu) / \sigma$$

dengan x adalah nilai asli, μ rata-rata fitur, dan σ simpangan baku fitur. Parameter μ dan σ hanya boleh dihitung dari data latih lalu diterapkan ke data uji agar tidak terjadi kebocoran data (data leakage).

2.4 Decision Tree

Decision Tree adalah algoritma klasifikasi yang membentuk model berupa struktur pohon: simpul internal merepresentasikan pengujian terhadap suatu atribut, cabang merepresentasikan hasil pengujian, dan simpul daun menyimpan keputusan kelas (Dwinnie dkk., 2024; Charbuty & Abdulazeez, 2021). Proses pembentukan pohon dilakukan dengan membagi data secara rekursif berdasarkan atribut yang paling baik memisahkan kelas. Kualitas pemisahan diukur dengan ukuran ketidakmurnian (impurity), antara lain entropy dan information gain (Putri dkk., 2024):

$$Entropy(S) = -\Sigma\, p_i \log_2 p_i$$

$$Gain(S, A) = Entropy(S) - \Sigma\, (|S_v| / |S|) \times Entropy(S_v)$$

Selain entropy, ukuran lain yang digunakan adalah Gini impurity, yaitu $Gini(S) = 1 - \Sigma\, p_i^2$, yang menjadi kriteria bawaan pada implementasi CART di scikit-learn. Atribut dengan penurunan impurity terbesar dipilih sebagai simpul pemisah. Keunggulan Decision Tree adalah mudah diinterpretasikan, dapat menangani data numerik maupun kategorikal, dan tidak memerlukan penskalaan fitur. Kelemahannya adalah mudah overfitting apabila pohon dibiarkan tumbuh terlalu dalam (Putri dkk., 2024). Untuk mengendalikannya digunakan hyperparameter seperti max_depth, min_samples_split, dan min_samples_leaf. Decision Tree juga menghasilkan feature importance, yaitu ukuran kontribusi tiap fitur dalam menurunkan impurity di seluruh pohon.

2.5 K-Nearest Neighbors (KNN)

K-Nearest Neighbors adalah algoritma klasifikasi berbasis instance yang memprediksi kelas suatu data baru berdasarkan kelas mayoritas dari k tetangga terdekatnya pada data latih (Uddin dkk., 2022). KNN termasuk lazy learning, karena tidak membangun model eksplisit pada tahap pelatihan dan baru melakukan perhitungan saat klasifikasi (Putri dkk., 2024). Langkah kerjanya adalah menentukan nilai k, menghitung jarak data uji ke seluruh data latih, mengurutkan jarak, mengambil

k tetangga terdekat, lalu menentukan kelas mayoritas. Ukuran jarak yang paling umum adalah Euclidean distance, sedangkan alternatifnya adalah Manhattan distance:

$$d_euclidean(p, q) = \sqrt{(\Sigma (p_i - q_i)^2)} \qquad d_manhattan(p, q) = \Sigma |p_i - q_i|$$

Nilai k merupakan bilangan bulat positif yang harus ditentukan sebelum algoritma dijalankan, dan penentuannya menjadi salah satu kelemahan KNN, selain pemilihan ukuran jarak dan seleksi fitur (Dwinnie dkk., 2024). Allorerung dkk. (2024) menyebutkan dua cara menentukan k, yaitu memakai bilangan ganjil kecil atau rumus k = √N, dengan N jumlah data latih. Pada praktiknya, k dioptimalkan dengan validasi silang. Nilai k yang terlalu kecil membuat model sensitif terhadap noise (varians tinggi), sedangkan k yang terlalu besar membuat batas keputusan terlalu halus (bias tinggi). Variasi lain adalah pembobotan jarak (weights = distance), yaitu tetangga yang lebih dekat diberi pengaruh lebih besar. Karena sepenuhnya bergantung pada jarak, KNN sangat dipengaruhi skala fitur sehingga standarisasi menjadi langkah wajib, dan KNN cenderung rentan pada data berdimensi tinggi (Allorerung dkk., 2024; Putri dkk., 2024).

2.6 Evaluasi Model

Kinerja model klasifikasi dievaluasi menggunakan confusion matrix, yaitu tabel yang memuat jumlah data uji yang diklasifikasikan benar dan salah. Tabel ini memiliki empat nilai: True Positive (TP), True Negative (TN), False Positive (FP), dan False Negative (FN) (Dwinnie dkk., 2024). Dari keempat nilai tersebut dihitung metrik berikut.

$$Akurasi = (TP + TN) / (TP + TN + FP + FN) \qquad Precision = TP / (TP + FP)$$

$$Recall = TP / (TP + FN) \qquad F1\text{-}score = 2 \times (Precision \times Recall) / (Precision + Recall)$$

Akurasi menunjukkan persentase data yang diklasifikasikan benar (Allorerung dkk., 2024). Precision mengukur ketepatan prediksi positif, recall mengukur kemampuan model menemukan seluruh data positif, dan F1-score adalah rata-rata harmonik keduanya. Untuk memperoleh estimasi performa yang lebih stabil digunakan stratified k-fold cross validation, dan pencarian hyperparameter terbaik dilakukan dengan Grid Search yang menguji seluruh kombinasi parameter menggunakan validasi silang pada data latih.

2.7 Penelitian Terkait

Beberapa penelitian telah membandingkan Decision Tree dan KNN dengan hasil yang bervariasi bergantung pada karakteristik data. Putri dkk. (2024) pada klasifikasi risiko obesitas melaporkan akurasi Decision Tree sebesar 84,98% dan KNN sebesar 83,55%. Dwinnie dkk. (2024) pada klasifikasi risiko diabetes memperoleh akurasi yang sama, yaitu 86%, untuk Decision Tree dan KNN. Suryadi dan Murhaban (2023) pada klasifikasi penyakit daun kopi memperoleh akurasi

Decision Tree 98,20% jauh di atas KNN 75,01%, sedangkan Zulkarnain dan Azmi (2024) pada klasifikasi rating Netflix menemukan Naive Bayes lebih unggul dibanding Decision Tree dan KNN. Temuan-temuan ini menunjukkan bahwa tidak ada algoritma yang selalu terbaik, sehingga perbandingan pada dataset yang digunakan perlu dilakukan secara empiris.

BAB III
HASIL

Seluruh proses dikerjakan menggunakan bahasa Python dengan pustaka pandas, numpy, matplotlib, seaborn, dan scikit-learn. Pada bab ini dijelaskan hasil pemahaman data, EDA, preprocessing, pelatihan Decision Tree dan KNN, serta perbandingan akurasi kedua model beserta analisisnya.

3.1 Pemahaman Dataset

Dataset terdiri dari 2.000 baris dan 14 kolom: 11 kolom numerik, 1 kolom boolean (bomb_planted), dan 2 kolom teks (map dan round_winner). Pemeriksaan dengan isnull().sum() menunjukkan tidak ada data kosong pada seluruh kolom, sedangkan duplicated().sum() menemukan 39 baris duplikat. Deskripsi atribut disajikan pada Tabel 3.1 dan statistik deskriptif atribut numerik pada Tabel

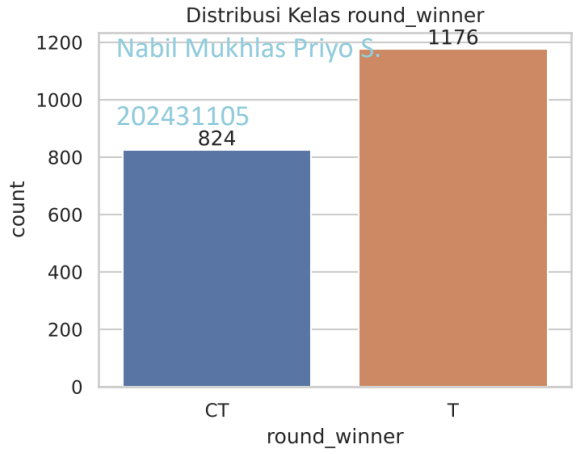
Atribut	Tipe	Keterangan
time_left	Numerik	Sisa waktu ronde dalam detik (0,05 – 175)
ct_score, t_score	Numerik	Skor sementara tim CT dan T (0 – 15)
map	Kategorikal	Peta permainan (7 peta: de_mirage, de_nuke, de_dust2, de_inferno, de_overpass, de_vertigo, de_train)
bomb_planted	Boolean	Status bom sudah dipasang (True = 264 baris, False = 1.736 baris)
ct_health, t_health	Numerik	Total nyawa pemain yang masih hidup per tim (0 – 500)
ct_armor, t_armor	Numerik	Total armor per tim (0 – 500)
ct_money, t_money	Numerik	Total uang per tim (maksimum 58.050 untuk CT dan 67.700 untuk T)
ct_helmets, t_helmets	Numerik	Jumlah pemain yang memakai helm per tim (0 – 5)
round_winner	Kategorikal (target)	Tim pemenang ronde: CT atau T

Atribut	Rata-rata	Std. Deviasi	Min	Maks
time_left	95,63	53,73	0,05	175,00
ct_score	6,84	4,23	0	15
t_score	6,29	4,19	0	15
ct_health	406,17	137,62	2	500
t_health	402,99	137,70	0	500
ct_armor	306,75	172,86	0	500
t_armor	309,54	168,69	0	500
ct_money	7.867,35	9.017,49	0	58.050
t_money	12.377,60	12.209,58	0	67.700
ct_helmets	1,82	1,77	0	5
t_helmets	3,01	1,96	0	5

Tabel memperlihatkan perbedaan skala yang sangat besar antar atribut: atribut uang (money) bernilai hingga puluhan ribu, sedangkan skor dan jumlah helm hanya berkisar 0 – 15 dan 0 – 5. Perbedaan skala ini menjadi alasan utama dilakukannya standarisasi sebelum melatih KNN. Rata-rata t_money (12.377,60) juga jauh lebih besar dibanding ct_money (7.867,35), yang mengindikasikan tim T cenderung memiliki sumber daya lebih besar pada data ini.

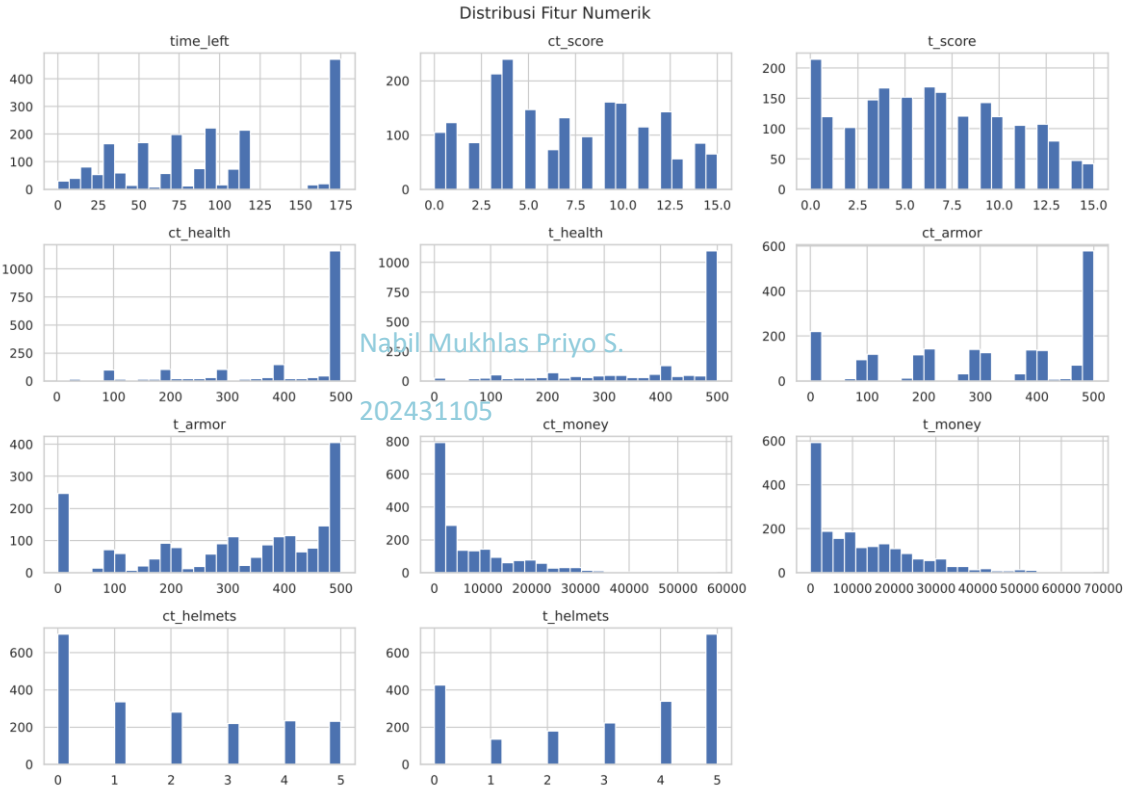
3.2 Exploratory Data Analysis

3.2.1 Distribusi Kelas Target



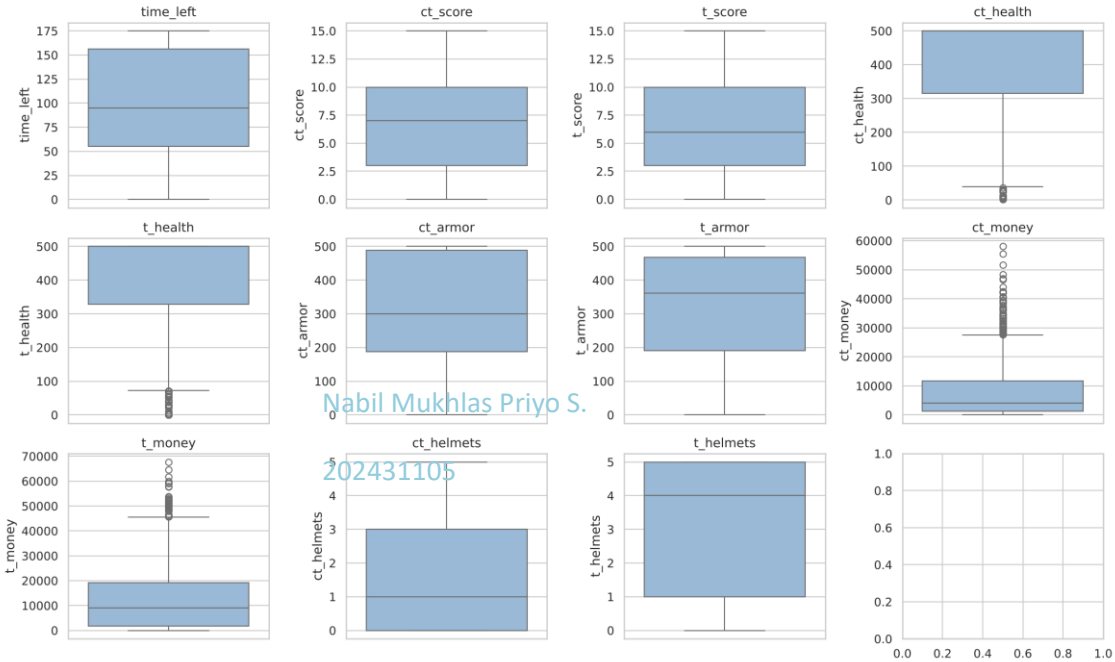
Gambar menunjukkan bahwa kelas T berjumlah 1.176 baris (58,8%) dan kelas CT berjumlah 824 baris (41,2%). Data bersifat tidak seimbang secara moderat, dengan rasio sekitar 1,4 : 1. Dengan proporsi ini, model yang selalu menebak kelas T akan mencapai akurasi sekitar 58,8%, sehingga angka tersebut menjadi baseline minimal yang harus dilampaui oleh model. Karena itu, selain akurasi, precision, recall, dan F1-score juga dilaporkan, dan pembagian data dilakukan secara stratified.

3.2.2 Distribusi dan Outlier Fitur Numerik



Nabil Mukhlas Priyo S.
202431105

Histogram pada Gambar memperlihatkan bahwa ct_health dan t_health menumpuk pada nilai 500, yaitu kondisi saat seluruh lima pemain masih hidup, sehingga distribusinya sangat condong ke kanan. Atribut ct_money dan t_money juga condong ke kanan karena banyak ronde dengan uang rendah dan sebagian kecil dengan uang sangat besar. Atribut time_left, ct_score, dan t_score menyebar relatif merata, sedangkan t_helmets terkonsentrasi pada nilai tinggi (4 – 5) dan ct_helmets pada nilai rendah.

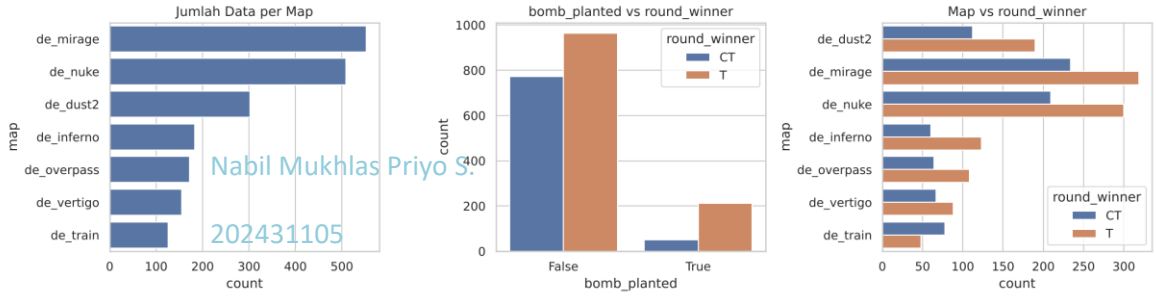


Nabil Mukhlas Priyo S.
202431105

Atribut	Jumlah outlier	Atribut	Jumlah outlier
ct_health	24	ct_money	90
t_health	53	t_money	45
Atribut lain (time_left, skor, armor, helm)	0		

Outlier hanya ditemukan pada atribut health dan money. Nilai-nilai tersebut tidak dihapus karena merupakan kondisi permainan yang sah, misalnya sebagian besar pemain sudah gugur sehingga health rendah, atau tim menabung dalam beberapa ronde sehingga uangnya sangat besar. Menghapusnya justru dapat membuang situasi penting yang menentukan pemenang ronde. Decision Tree tidak terpengaruh oleh outlier karena hanya menggunakan ambang pemisah, sedangkan pengaruhnya pada KNN diredam oleh standarisasi.

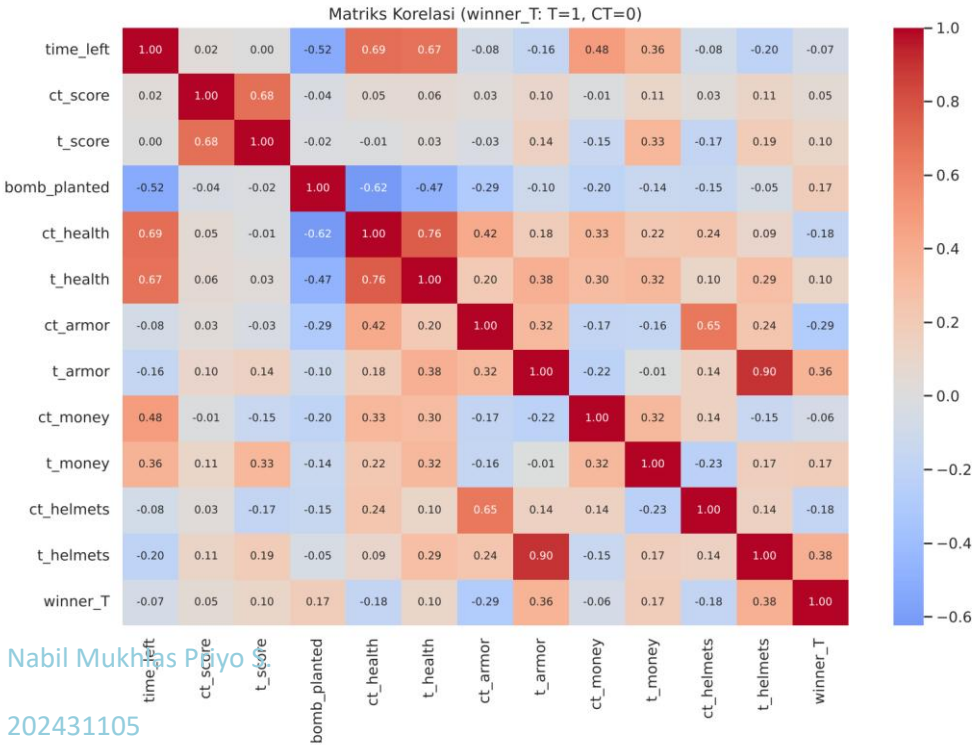
3.2.3 Analisis Atribut Kategorikal



Nabil Mukhlas Priyo S.
202431105

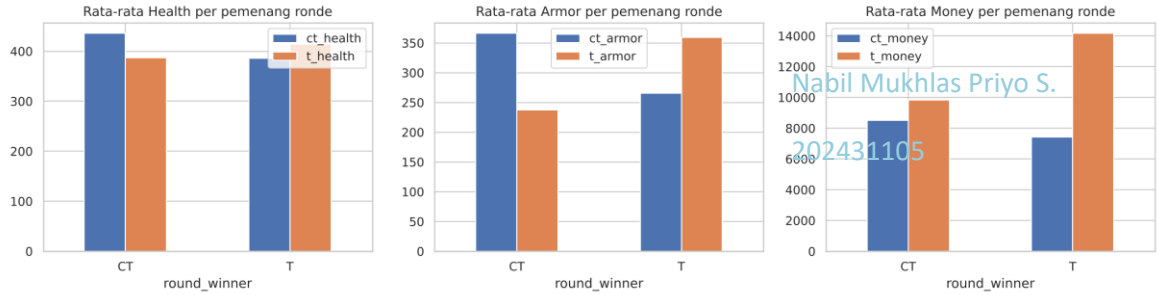
Peta de_mirage (553 baris) dan de_nuke (509 baris) paling banyak muncul, sedangkan de_train paling sedikit (126 baris). Kemenangan tim T mendominasi hampir semua peta, kecuali de_train yang dimenangkan CT sebesar 61,9% (hanya 126 baris sehingga kurang representatif). Pengaruh status bom jauh lebih kuat: ketika bom sudah dipasang (264 baris), tim T menang pada 80,7% ronde, sedangkan ketika bom belum dipasang tim T menang pada 55,5% ronde. Hal ini masuk akal karena bom yang sudah terpasang memaksa CT harus menjinakkannya untuk menang.

3.2.4 Korelasi Antar Fitur dan Perbandingan Rata-rata



Nabil Mukhlas Priyo S.
202431105

Berdasarkan Gambar 3.5, korelasi tertinggi dengan kemenangan T adalah t_helmets (+0,376), t_armor (+0,356), t_money (+0,175), dan bomb_planted (+0,173). Sebaliknya, ct_armor (−0,287), ct_health (−0,179), dan ct_helmets (−0,177) berkorelasi negatif, yang berarti semakin kuat perlengkapan CT, semakin kecil peluang T menang. Korelasi tertinggi ini bernilai sedang, sehingga tidak ada satu fitur pun yang cukup untuk memprediksi pemenang secara tunggal. Hubungan antar fitur kemungkinan bersifat gabungan dan tidak linear, dan kondisi ini cocok untuk model non-linear seperti Decision Tree dan KNN.



Nabil Mukhlas Priyo S.
202431105

Gambar mendukung hasil korelasi tersebut. Pada ronde yang dimenangkan T, rata-rata t_armor sebesar 359,8 dan rata-rata t_money sebesar 14.162, jauh lebih tinggi dibanding ronde yang dimenangkan CT (t_armor 237,8 dan t_money 9.831). Pada ronde yang dimenangkan CT, rata-rata ct_armor sebesar 366,0 dibanding 265,2 pada ronde yang dimenangkan T. Dengan demikian armor dan money merupakan kandidat fitur yang paling membedakan kedua kelas.

3.3 Preprocessing Data

Tahapan preprocessing yang dilakukan dirangkum pada Tabel, dan potongan kodenya ditunjukkan di bawah tabel.

No	Tahapan	Perlakuan	Hasil
1	Missing value	isnull().sum()	Tidak ada data kosong, tidak diperlukan imputasi
2	Data duplikat	drop_duplicates()	39 baris dihapus, data menjadi 1.961 baris (T = 1.156, CT = 805)
3	Encoding bomb_planted	astype(int)	False/True diubah menjadi 0/1
4	Encoding map	One-hot (get_dummies)	1 kolom menjadi 7 kolom biner, total fitur = 19
5	Encoding target	LabelEncoder	CT = 0 dan T = 1
6	Split data	train_test_split 80:20, stratify, random_state = 42	Data latih 1.568 baris, data uji 393 baris
7	Standarisasi	StandardScaler (fit pada data latih)	Fitur berskala rata-rata 0 dan std 1, khusus untuk KNN

```
data = df.drop_duplicates().copy()
data["bomb_planted"] = data["bomb_planted"].astype(int)
data = pd.get_dummies(data, columns=["map"], dtype=int)
y = LabelEncoder().fit_transform(data["round_winner"])   # CT=0, T=1
X = data.drop(columns=["round_winner"])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)   # fit hanya pada data latih
X_test_s  = scaler.transform(X_test)
```

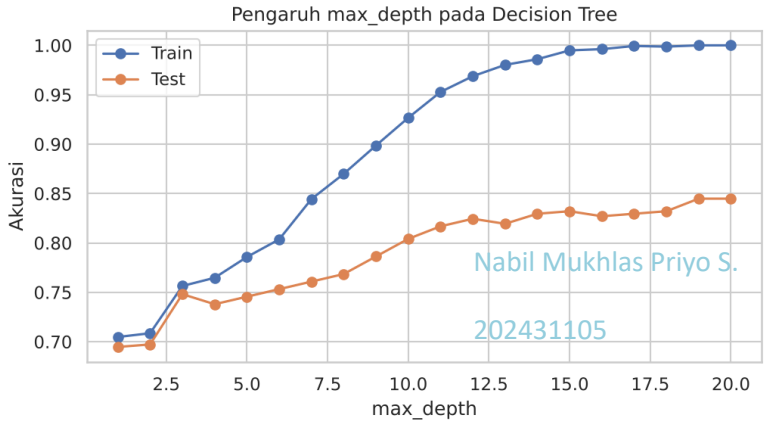
Kode di atas menghapus baris duplikat, mengubah bomb_planted menjadi 0/1, dan melakukan one-hot encoding pada map. Label target kemudian diubah menjadi angka dengan CT = 0 dan T = 1 sehingga kelas T menjadi kelas positif dalam perhitungan precision, recall, dan F1-score. Data dibagi 80:20 dengan stratify sehingga proporsi kelas pada data latih dan uji sama dengan data asli. StandardScaler di-fit hanya pada data latih lalu diterapkan ke data uji agar tidak terjadi kebocoran informasi. Hasil standarisasi hanya dipakai oleh KNN karena Decision Tree tidak terpengaruh oleh skala fitur, sehingga Decision Tree dilatih menggunakan data tanpa standarisasi.

3.4 Pelatihan dan Hasil Decision Tree

```
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
pred = dt.predict(X_test)
print(accuracy_score(y_test, pred))

param_dt = {"max_depth": [3, 5, 7, 10, 15, None],
            "min_samples_split": [2, 5, 10, 20],
            "min_samples_leaf": [1, 2, 5, 10],
            "criterion": ["gini", "entropy"]}
cv = StratifiedKFold(5, shuffle=True, random_state=42)
gs = GridSearchCV(DecisionTreeClassifier(random_state=42),
                  param_dt, cv=cv, scoring="accuracy", n_jobs=-1)
gs.fit(X_train, y_train)
dt_best = gs.best_estimator_
```

Model pertama adalah Decision Tree dengan parameter bawaan, yang kemudian dibandingkan dengan model hasil Grid Search. Grid Search menguji 192 kombinasi parameter (criterion, max_depth, min_samples_split, dan min_samples_leaf) dengan stratified 5-fold cross validation pada data latih saja, sehingga data uji tidak ikut memengaruhi pemilihan parameter.

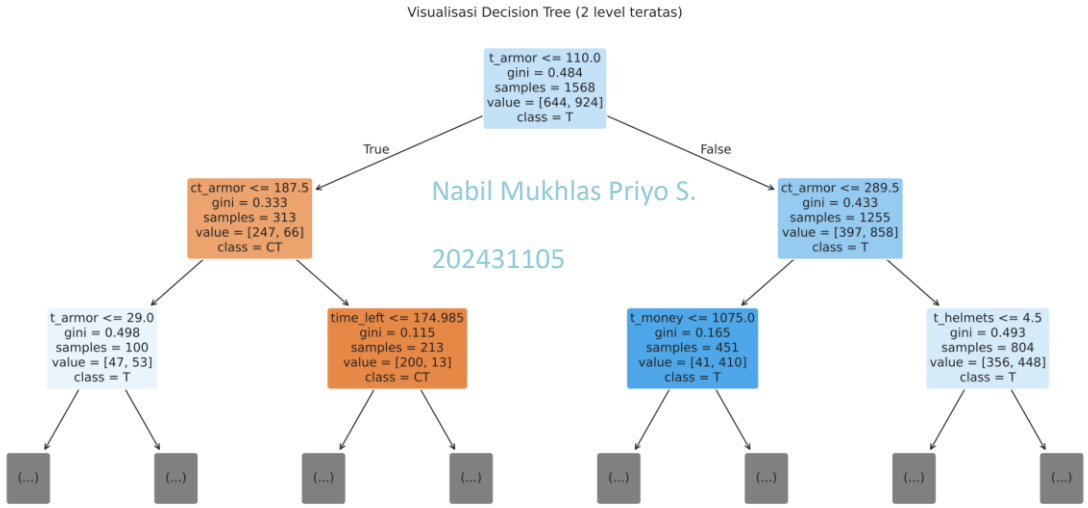


Gambar menunjukkan pengaruh kedalaman pohon. Akurasi data uji naik dari sekitar 69% pada kedalaman 1 menjadi sekitar 84% pada kedalaman 20, sementara akurasi data latih mendekati 100% pada kedalaman sekitar 15 ke atas. Jarak yang lebar antara kedua kurva menandakan overfitting: pohon yang dalam menghafal data latih. Namun akurasi data uji tidak turun ketika pohon diperdalam, melainkan terus naik secara perlahan. Hal ini menjelaskan hasil Grid Search berikut.

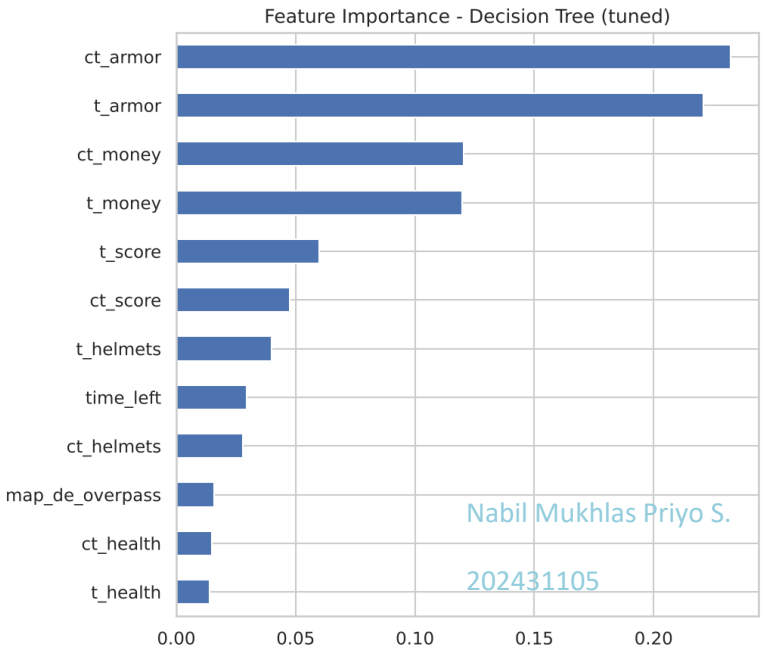
Parameter	Ruang pencarian	Terbaik
criterion	gini, entropy	gini
max_depth	3, 5, 7, 10, 15, None	None (tanpa batas)
min_samples_split	2, 5, 10, 20	2
min_samples_leaf	1, 2, 5, 10	1
Akurasi cross validation (data latih)		82,40%

Kombinasi terbaik ternyata sama dengan parameter bawaan, yaitu pohon tanpa pembatasan kedalaman dengan kriteria Gini. Artinya, pemangkasan pohon (pruning) melalui max_depth,

min_samples_split, maupun min_samples_leaf tidak meningkatkan akurasi pada data ini. Pola data tampak rinci dan membutuhkan banyak pemisahan, sehingga pembatasan pohon justru menurunkan kemampuan prediksi. Oleh karena itu, hasil Decision Tree bawaan dan Decision Tree hasil tuning identik.



Gambar menunjukkan simpul akar memisahkan data berdasarkan t_armor ≤ 110. Jika armor tim T rendah (313 baris), CT menang pada 247 baris (78,9%). Jika armor T lebih tinggi (1.255 baris), cabang berikutnya memeriksa ct_armor ≤ 289,5. Pada kelompok dengan t_armor > 110 dan ct_armor ≤ 289,5 (451 baris), tim T menang pada 410 baris (90,9%). Aturan-aturan ini sejalan dengan hasil EDA bahwa armor dan money adalah penentu utama.



Feature importance pada Gambar menunjukkan ct_armor (0,232), t_armor (0,221), ct_money (0,120), dan t_money (0,120) sebagai empat fitur terpenting dengan total kontribusi sekitar 69%. Skor

kedua tim (t_score 0,060 dan ct_score 0,048) berada di tingkat berikutnya, dan fitur lainnya, termasuk map, health, dan helm, memberi kontribusi kecil. Rendahnya kontribusi health kemungkinan disebabkan nilainya yang menumpuk pada 500 di banyak baris sehingga kurang membedakan pemenang.

Kelas	Precision	Recall	F1-score	Jumlah data
CT (0)	0,81	0,81	0,81	161
T (1)	0,87	0,87	0,87	232
Akurasi			0,8448	393

Decision Tree menghasilkan akurasi data uji sebesar 84,48% (precision 0,8701, recall 0,8664, F1-score 0,8683 untuk kelas T), sedangkan akurasi data latih mencapai 100%. Selisih sekitar 15,5 poin tersebut mengonfirmasi bahwa model overfitting. Selain itu, dengan stratified 5-fold cross validation pada seluruh data diperoleh akurasi rata-rata 85,31%, yang menunjukkan hasil pada data uji konsisten. Confusion matrix model disajikan bersama KNN pada Gambar.

3.5 Pelatihan dan Hasil KNN

```
knn_raw = KNeighborsClassifier(n_neighbors=5).fit(X_train, y_train)     # tanpa scaling
knn_s   = KNeighborsClassifier(n_neighbors=5).fit(X_train_s, y_train)   # dengan scaling

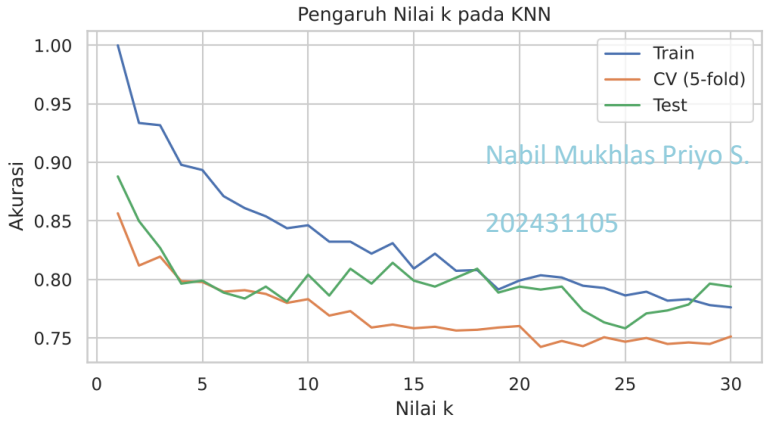
param_knn = {"n_neighbors": list(range(1, 31)),
             "weights": ["uniform", "distance"],
             "metric": ["euclidean", "manhattan"]}
gs_knn = GridSearchCV(KNeighborsClassifier(), param_knn,
                      cv=cv, scoring="accuracy", n_jobs=-1)
gs_knn.fit(X_train_s, y_train)
knn_best = gs_knn.best_estimator_
pred = knn_best.predict(X_test_s)
```

KNN dilatih menggunakan data yang sudah distandarisasi. Sebagai pembanding, KNN dengan k = 5 juga dilatih pada data tanpa standarisasi untuk melihat pengaruh penskalaan. Kemudian, Grid Search menguji 120 kombinasi (k = 1 sampai 30, dua jenis pembobotan, dan dua ukuran jarak) dengan 5-fold cross validation pada data latih.

Skenario	Akurasi latih	Akurasi uji	Precision (T)	Recall (T)	F1 (T)
Tanpa standarisasi	82,53%	72,26%	0,7531	0,7888	0,7705
Dengan standarisasi	89,35%	79,90%	0,8148	0,8534	0,8337

Standarisasi menaikkan akurasi uji KNN dari 72,26% menjadi 79,90%, yaitu peningkatan 7,64 poin tanpa mengubah algoritma atau parameter. Hasil ini sesuai dengan teori bahwa algoritma berbasis jarak bergantung pada skala fitur: tanpa standarisasi, jarak Euclidean didominasi ct_money dan t_money yang bernilai puluhan ribu, sedangkan fitur penting lain seperti skor dan helm hampir

tidak berpengaruh. Temuan ini sejalan dengan Allorerung dkk. (2024) yang melaporkan akurasi rendah pada KNN tanpa normalisasi.



Gambar memperlihatkan bahwa akurasi KNN cenderung menurun ketika k membesar. Akurasi cross validation turun dari 85,65% pada k = 1 menjadi sekitar 75% pada k = 30. Pola ini menunjukkan bahwa kelas pada dataset ditentukan oleh tetangga yang sangat dekat (struktur lokal), sehingga perataan suara dari banyak tetangga justru mengaburkan batas kelas. Nilai k kecil ideal untuk data ini, tetapi k = 1 sensitif terhadap noise sehingga hasil Grid Search memilih k = 3.

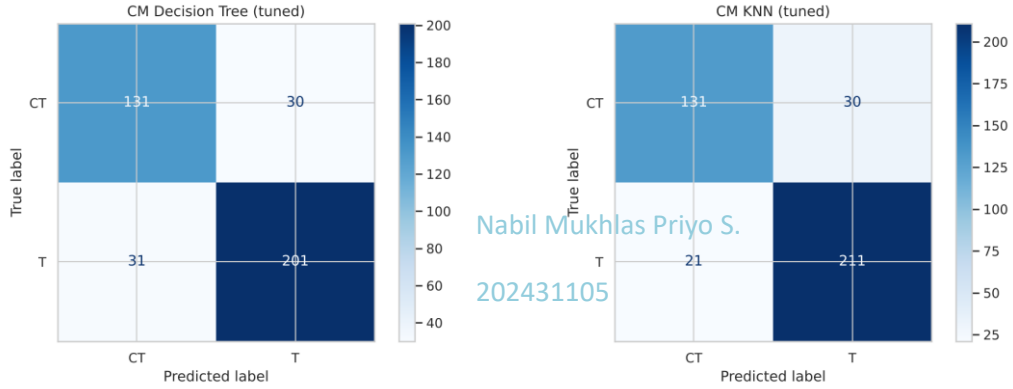
Parameter	Ruang pencarian	Terbaik
n_neighbors (k)	1 sampai 30	3
weights	uniform, distance	distance
metric	euclidean, manhattan	manhattan
Akurasi cross validation (data latih)		88,14%

Model KNN terbaik memakai k = 3, pembobotan berdasarkan jarak, dan jarak Manhattan. Pada data uji, model ini mencapai akurasi 87,02% dengan precision 0,8755, recall 0,9095, dan F1-score 0,8922 untuk kelas T. Akurasi data latih sebesar 100% pada model ini tidak berarti overfitting yang sama seperti pada Decision Tree: dengan weights = distance, setiap data latih menjadi tetangga terdekat bagi dirinya sendiri dengan jarak nol sehingga selalu diprediksi benar. Karena itu, akurasi data uji dan cross validation adalah ukuran yang lebih tepat. Dengan stratified 5-fold cross validation pada seluruh data, KNN terbaik memperoleh akurasi rata-rata 88,27%.

Laporan klasifikasi KNN terbaik pada data uji (393 baris)

Kelas	Precision	Recall	F1-score	Jumlah data
CT (0)	0,86	0,81	0,84	161
T (1)	0,88	0,91	0,89	232
Akurasi			0,8702	393

3.6 Perbandingan Model dan Analisis

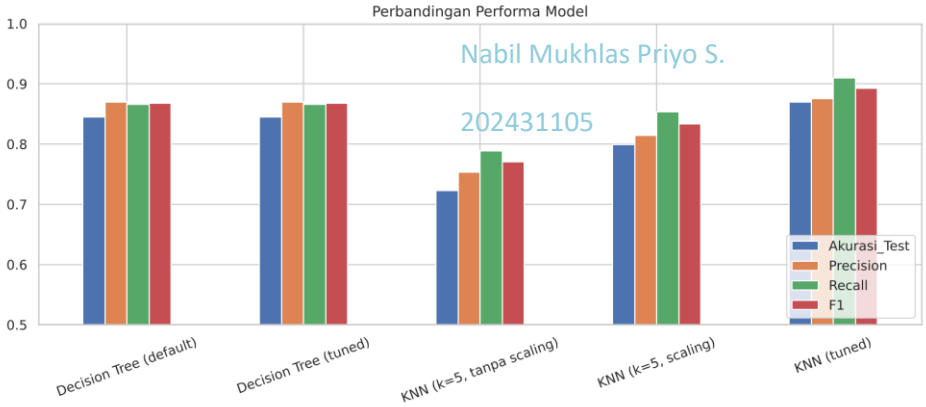


Confusion matrix data uji: (kiri) Decision Tree, (kanan) KNN terbaik

Pada Decision Tree, dari 161 data CT sebanyak 131 diprediksi benar dan 30 salah menjadi T, sedangkan dari 232 data T sebanyak 201 benar dan 31 salah menjadi CT. Pada KNN terbaik, kelas CT menghasilkan hasil yang sama (131 benar dan 30 salah), tetapi dari 232 data T sebanyak 211 diprediksi benar dan hanya 21 salah. Jadi, keunggulan KNN terutama berasal dari kemampuannya mengenali kelas T (recall naik dari 0,8664 menjadi 0,9095, atau 10 data lebih banyak yang terdeteksi), sedangkan kemampuan mengenali kelas CT sama persis pada kedua model.

Perbandingan seluruh model (kelas positif = T)

Model	Akurasi latih	Akurasi uji	Precision	Recall	F1-score
Decision Tree (default)	100,00%	84,48%	0,8701	0,8664	0,8683
Decision Tree (tuned)	100,00%	84,48%	0,8701	0,8664	0,8683
KNN (k=5, tanpa scaling)	82,53%	72,26%	0,7531	0,7888	0,7705
KNN (k=5, scaling)	89,35%	79,90%	0,8148	0,8534	0,8337
KNN (tuned)	100,00%	87,02%	0,8755	0,9095	0,8922



Perbandingan akurasi uji, precision, recall, dan F1-score seluruh model

Akurasi stratified 5-fold cross validation pada seluruh data

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Rata-rata
Decision Tree	85,75%	86,48%	83,16%	85,71%	85,46%	85,31%
KNN (tuned)	87,53%	86,48%	88,52%	89,03%	89,80%	88,27%

Berdasarkan Tabel, akurasi tertinggi pada data uji diperoleh KNN terbaik sebesar 87,02%, unggul 2,54 poin dibanding Decision Tree sebesar 84,48%. Keunggulan ini konsisten pada validasi silang (Tabel 3.11): rata-rata KNN 88,27% berbanding 85,31% untuk Decision Tree, dengan akurasi KNN lebih tinggi atau sama pada kelima fold. Seluruh model melampaui baseline 58,8% (menebak kelas T), sehingga kedua algoritma terbukti mempelajari pola yang bermakna.

Beberapa faktor dapat menjelaskan hasil tersebut. Pertama, seluruh fitur bersifat numerik berdimensi rendah (19 fitur setelah encoding) dan sudah distandarisasi, kondisi yang mendukung KNN karena jarak antar data bermakna dan KNN rentan pada dimensi tinggi (Putri dkk., 2024). Kedua, hasil Grid Search memilih k kecil (k = 3), yang berarti kelas ditentukan oleh data yang sangat mirip. Ketiga, Decision Tree mengalami overfitting yang tidak dapat diatasi oleh pruning pada data ini, dan hasilnya sulit dinaikkan melalui tuning. Keempat, KNN sangat bergantung pada standarisasi (72,26% menjadi 79,90%) dan penyetelan parameter, sehingga akurasinya meningkat dari 79,90% menjadi 87,02% setelah tuning, sedangkan Decision Tree tidak berubah.

Hasil ini berbeda dengan sebagian penelitian terkait. Putri dkk. (2024) dan Suryadi dan Murhaban (2023) menemukan Decision Tree lebih akurat dibanding KNN, sedangkan Dwinnie dkk. (2024) menemukan keduanya setara. Perbedaan tersebut wajar karena kinerja algoritma bergantung pada karakteristik data, seperti jenis fitur, dimensi, noise, serta perlakuan preprocessing dan tuning yang diterapkan. Hasil pada dataset ini menunjukkan bahwa dengan standarisasi dan tuning yang tepat, KNN dapat mengungguli Decision Tree.

Terdapat beberapa keterbatasan yang perlu diperhatikan. Pertama, dataset tampaknya berisi beberapa potret kondisi dari ronde yang sama (misalnya baris awal dengan skor 0–0 yang hanya berbeda pada time_left). Jika potret dari ronde yang sama tersebar di data latih dan data uji, akurasi dapat sedikit lebih optimis, terutama untuk KNN dengan k kecil yang mengandalkan data serupa. Dataset tidak memiliki kolom identitas ronde sehingga hal ini tidak dapat diverifikasi. Kedua, data uji hanya 393 baris sehingga selisih 1 poin persentase setara sekitar 4 data, dan selisih 2,54 poin antar model setara sekitar 10 data. Ketiga, data tidak seimbang (58,8% T), sehingga recall kelas CT (0,81) lebih rendah dibanding kelas T.

BAB IV
PENUTUP

4.1 Kesimpulan

Berdasarkan landasan teori dan hasil praktikum yang telah dilakukan, diperoleh kesimpulan sebagai berikut.

1. Dataset Counter-Strike 2 terdiri dari 2.000 baris dan 14 atribut tanpa data kosong, dengan 39 baris duplikat. Kelas target tidak seimbang moderat (T 58,8% dan CT 41,2%). Hasil EDA menunjukkan armor, uang, dan helm tim T serta status bom adalah faktor yang paling terkait dengan kemenangan T.
2. Preprocessing yang dilakukan meliputi penghapusan duplikat (menjadi 1.961 baris), encoding bomb_planted, map, dan target, pembagian data 80:20 secara stratified (1.568 data latih dan 393 data uji), serta standarisasi fitur untuk KNN. Standarisasi terbukti penting karena menaikkan akurasi KNN dari 72,26% menjadi 79,90%.
3. Decision Tree menghasilkan akurasi uji 84,48% (F1-score 0,8683), baik dengan parameter bawaan maupun setelah Grid Search. Model ini overfitting (akurasi latih 100%), tetapi pembatasan kedalaman pohon tidak meningkatkan akurasi. Fitur terpenting adalah ct_armor, t_armor, ct_money, dan t_money.
4. KNN terbaik (k = 3, weights = distance, metric = manhattan) menghasilkan akurasi uji 87,02% (F1-score 0,8922) dan akurasi cross validation 88,27%, lebih tinggi dibanding Decision Tree (84,48% dan 85,31%). Jadi, KNN menjadi model terbaik pada dataset ini, terutama karena kemampuannya mengenali kelas T lebih baik (recall 0,9095).
5. Hasil ini menunjukkan bahwa pemilihan algoritma, preprocessing (khususnya penskalaan pada KNN), dan penyetelan hyperparameter sangat memengaruhi akurasi, serta tidak ada algoritma yang selalu unggul pada semua data.

4.2 Saran

Untuk pengembangan selanjutnya disarankan: (1) menambahkan identitas ronde atau melakukan pembagian data berdasarkan ronde (group split) agar evaluasi lebih bebas dari kemungkinan kebocoran antar potret ronde yang sama; (2) menangani ketidakseimbangan kelas dengan teknik seperti class_weight atau SMOTE; (3) membandingkan dengan algoritma lain seperti Random Forest, Gradient Boosting, atau SVM; dan (4) menambahkan fitur turunan, misalnya selisih armor dan uang antar tim, untuk meningkatkan akurasi.

DAFTAR PUSTAKA

Allorerung, P. P., Erna, A., Bagussahrir, M., & Alam, S. (2024). Analisis performa normalisasi data untuk klasifikasi K-Nearest Neighbor pada dataset penyakit. *JISKA (Jurnal Informatika Sunan Kalijaga), 9*(3), 178–191.

Charbuty, B., & Abdulazeez, A. (2021). Classification based on decision tree algorithm for machine learning. *Journal of Applied Science and Technology Trends, 2*(1), 20–28. https://doi.org/10.38094/jastt20165

Dwinnie, Z. C., Dwynne, Z. C., Islam, M. J., & Noviarni. (2024). Comparison of machine learning algorithms in diabetes risk classification. *IJATIS: Indonesian Journal of Applied Technology and Innovation Science, 1*(2), 54–60. https://doi.org/10.57152/ijatis.v1i2.1141

Putri, A. I., Husna, N. A., Cia, N. M., Arba, M. A., Aisyi, N. R., Pramesthi, C. H., & Irdayusman, A. S. (2024). Implementation of K-Nearest Neighbors, Naïve Bayes Classifier, Support Vector Machine and Decision Tree algorithms for obesity risk prediction. *PREDATECS: Public Research Journal of Engineering, Data Technology and Computer Science, 2*(1), 26–33. https://doi.org/10.57152/predatecs.v2i1.1110

Suryadi, & Murhaban. (2023). Comparative analysis of the performance of the Decision Tree and K-Nearest Neighbors methods in classifying coffee leaf diseases. *IICS (SEMNASTIK 2023), 4*(1).

Uddin, S., Haque, I., Lu, H., Moni, M. A., & Gide, E. (2022). Comparative performance analysis of K-nearest neighbour (KNN) algorithm and its different variants for disease prediction. *Scientific Reports, 12*, 1–11. https://doi.org/10.1038/s41598-022-10358-x

Zulkarnain, Z., & Azmi, H. (2024). Performance comparison K-Nearest Neighbor, Naive Bayes, and Decision Tree algorithms for Netflix rating classification. *IJATIS: Indonesian Journal of Applied Technology and Innovation Science, 1*(1), 16–22. https://doi.org/10.57152/ijatis.v1i1.1104